

Smart Leader: Loan Eligibility Prediction System

Project Description:

Smart Leader is a Machine Learning based web application designed to predict the creditworthiness of loan applicants, enabling banks and financial institutions to make faster, data-driven loan approval decisions. The platform evaluates classification algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost to determine the likelihood that an applicant will repay or default on a loan.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models, the best-performing model — XGBoost, achieving 94.7% training accuracy and 81.1% testing accuracy — is saved and integrated into a Flask web application for real-time predictions.

Built with Python and Flask and designed for deployment on IBM Cloud, Smart Leader provides a seamless interface where applicants submit their details and instantly receive an eligibility prediction. The system helps financial analysts, credit officers, and banking professionals reduce non-performing assets and improve the overall credit evaluation process with confidence.

Scenarios

Scenario 1: Fast-Track Approval for Low-Risk Applicants

A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

Scenario 2: High-Risk Applicant Detection

An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk based on the trained model, prompting further scrutiny or document verification before proceeding.

Scenario 3: Batch Evaluation for High-Volume Periods

A financial analyst uses the platform to batch-evaluate multiple applicants during a high-volume period. By relying on the XGBoost model's predictions, the analyst significantly reduces evaluation time while maintaining accuracy and compliance.

Technical Architecture:

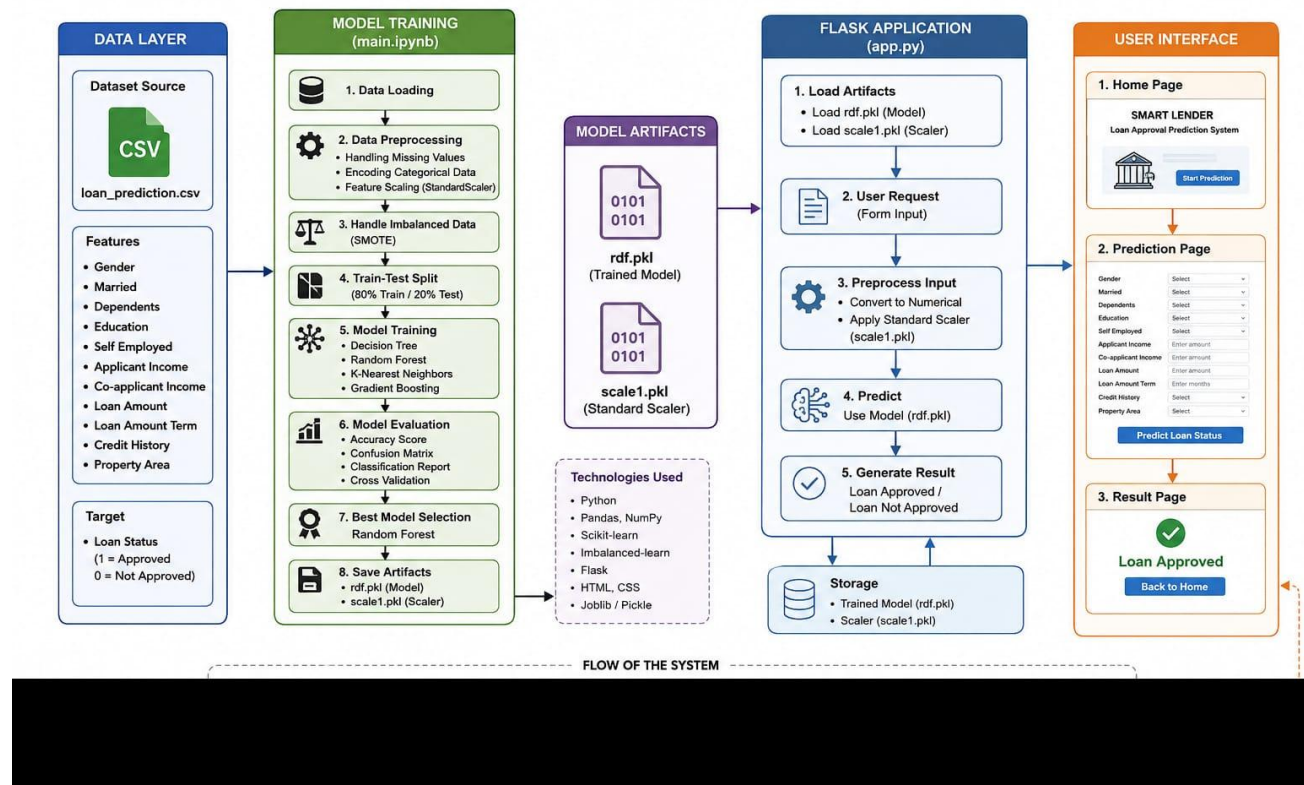
Smart Leader follows a modular pipeline architecture that carries applicant data from raw collection through preprocessing, model training, evaluation, and deployment:

- **Dataset Collection:** Applicant records are sourced from a Google Sheets dataset containing attributes such as gender, education, income, and loan amount.

Data Visualization & EDA: Matplotlib and Seaborn generate count plots, distribution plots, and bar charts to reveal trends and class distribution

SMART LENDER – LOAN APPROVAL PREDICTION SYSTEM

Technical Architecture



Preprocessing & Feature Engineering: Missing numerical values are filled using mean imputation, missing categorical values using mode imputation, and categorical variables are encoded into numerical format.

- Model Training: Decision Tree, Random Forest, KNN, and XGBoost classifiers are trained and evaluated on the processed dataset.
- Model Selection: The best-performing model (XGBoost) is saved for deployment.
- Flask Web Application: The saved model is integrated into a Flask app with a home page, a prediction interface, user input forms, and a result display.

(Note: If a database is used in the project, an ER Diagram along with a description should be added.)

Pre-requisites:

- Python Programming Proficiency: NumPy, Pandas
- Data Visualization: Matplotlib, Seaborn
- Machine Learning Fundamentals: Scikit-learn, SciPy, Classification Algorithms
- Flask Web Framework Knowledge
- HTML, CSS, and JavaScript Basics for the prediction interface
- Version Control with Git

Project Workflow:

Milestone 1: Environment Setup & Package Installation

- Activity 1.1: Install essential Python libraries and machine learning packages including NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn.
- Activity 1.2: Configure the environment to support data analysis, visualization, machine learning workflows, and Flask web application development.

Milestone 2: Dataset Collection & Understanding

- Activity 2.1: Download the loan eligibility dataset from the provided Google Sheets source.
- Activity 2.2: Analyze dataset features such as Gender, Education, Applicant Income, and Loan Amount.
- Activity 2.3: Understand relationships between applicant attributes and loan approval outcomes to identify factors influencing eligibility.

Milestone 3: Data Visualization & Analysis

- Activity 3.1: Perform exploratory data analysis (EDA) using Matplotlib and

Seaborn.

- Activity 3.2: Generate count plots, distribution plots, and bar charts to visualize class distributions and feature relationships.
- Activity 3.3: Analyze trends that impact loan approval outcomes.

Milestone 4: Data Preprocessing & Feature Engineering

- Activity 4.1: Handle missing values — fill numerical gaps using mean imputation and categorical gaps using mode imputation.
- Activity 4.2: Encode categorical variables into numerical format.
- Activity 4.3: Ensure the dataset is clean, balanced, and optimized for accurate model training.

Milestone 5: Machine Learning Model Building

- Activity 5.1: Train and evaluate Decision Tree, Random Forest, KNN, and XGBoost classifiers.
- Activity 5.2: Compare model performance and select the best-performing model (XGBoost) for deployment.

Milestone 6: Building the Flask Web Application

- Activity 6.1: Develop an interactive Flask-based web application with a home page introduction.
- Activity 6.2: Build the loan eligibility prediction interface with user input forms.
- Activity 6.3: Implement prediction result display, integrating the saved XGBoost model.

Milestone 7: Conclusion

Activity 1.1: Load and Explore the Dataset

The loan dataset is loaded into the Jupyter Notebook using the Pandas library.

The dataset contains historical records of loan applicants, including personal, financial, and loan-related information. The target variable is `Loan_Status`, which indicates whether a loan was approved or rejected.

Dataset exploration is performed using:

`head()` to display the first five records.

`shape` to determine the number of rows and columns.

`info()` to inspect data types and missing values.

`describe()` to obtain statistical summaries.

Data visualization techniques such as count plots, histograms, and heatmaps are used to understand feature distributions and relationships.

```
import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import DecisionTreeClassifier

data = pd.read_csv("../Dataset/loan_prediction.csv")
```

Python

Python

Activity 1.2: Data Cleaning and Feature Selection

The dataset is checked for missing values.

Missing numerical values are replaced using the mean, while missing categorical values are replaced using the mode.

The input features selected for prediction include:

Gender

Married

Dependents

Education

Self-Employed

Applicant Income

Co-applicant Income

Loan Amount

Loan Amount Term

Credit History

Property Area

The target variable is:

Loan_Status

```
data.head()
```

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0
LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0
LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0
LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0
LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Loan_ID              614 non-null    str
1   Gender               601 non-null    str
2   Married              611 non-null    str
3   Dependents           599 non-null    str
4   Education             614 non-null    str
5   Self_Employed        582 non-null    str
6   ApplicantIncome      614 non-null    int64
7   CoapplicantIncome    614 non-null    float64
8   LoanAmount           592 non-null    float64
9   Loan_Amount_Term     600 non-null    float64
10  Credit_History       564 non-null    float64
11  Property_Area        614 non-null    str
12  Loan_Status          614 non-null    str
dtypes: float64(4), int64(1), str(8)
memory usage: 62.5 KB
```

Activity 1.3: Data Preprocessing

Categorical variables are converted into numerical values using label encoding.

Examples include:

Male → 0

Female → 1

Yes → 1

No → 0

Property Area:

Rural → 0

Semiurban → 1

Urban → 2

Feature scaling is performed using StandardScaler to normalize numerical values and improve model performance.

The fitted scaler is saved as scale1.pkl for use during prediction.

```
# Handling missing values

data['Gender'] = data['Gender'].fillna(data['Gender'].mode()[0])
data['Married'] = data['Married'].fillna(data['Married'].mode()[0])
data['Dependents'] = data['Dependents'].str.replace('+', '', regex=False)
data['Dependents'] = data['Dependents'].fillna(data['Dependents'].mode()[0])
data['Self_Employed'] = data['Self_Employed'].fillna(data['Self_Employed'].mode()[0])
data['LoanAmount'] = data['LoanAmount'].fillna(data['LoanAmount'].mean())
data['Loan_Amount_Term'] = data['Loan_Amount_Term'].fillna(data['Loan_Amount_Term'].mode()[0])
data['Credit_History'] = data['Credit_History'].fillna(data['Credit_History'].mode()[0])

data.isnull().sum()

Loan_ID          0
Gender           0
Married          0
Dependents       0
Education        0
Self_Employed    0
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       0
Loan_Amount_Term 0
Credit_History   0
Property_Area    0
Loan_Status      0
dtype: int64
```

Activity 1.4: Model Training and Comparison

The dataset is divided into training and testing sets using the `train_test_split()` function.

Multiple machine learning algorithms are trained and evaluated, including:

Decision Tree

Random Forest

K-Nearest Neighbors (KNN)

Gradient Boosting

Each model is evaluated using:

Accuracy Score

Confusion Matrix

Classification Report

Cross Validation

After comparing all models, Random Forest achieved the highest prediction accuracy and overall performance.

```
# Building the Gradient Boosting model
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier(random_state=42)
gb.fit(X_train, y_train)

train_pred = gb.predict(X_train)
test_pred = gb.predict(X_test)

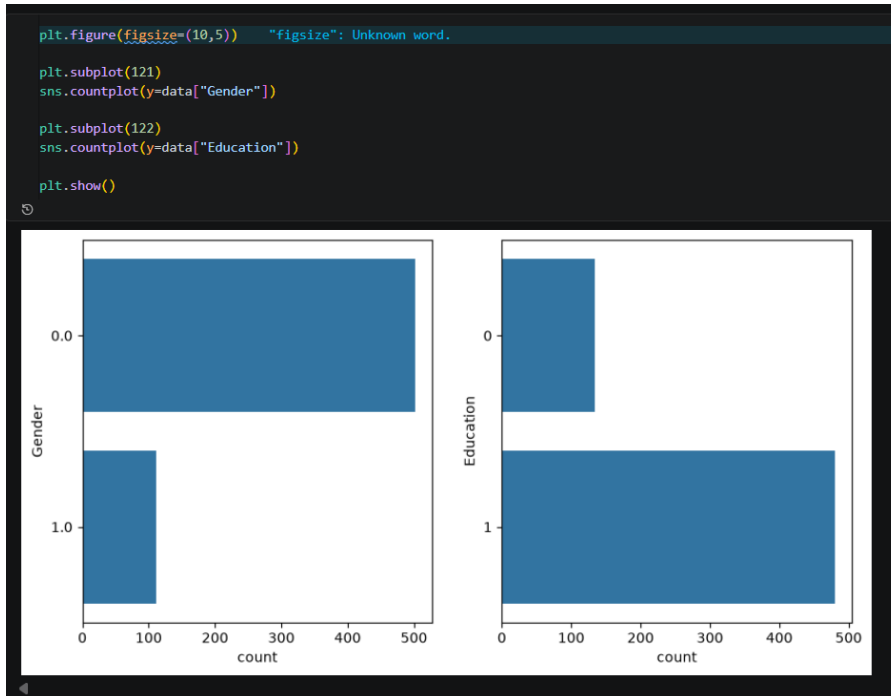
print("Training Accuracy :", accuracy_score(y_train, train_pred))
print("Testing Accuracy :", accuracy_score(y_test, test_pred))
```

```
Training Accuracy : 0.9099756690997567
Testing Accuracy : 0.7684729064039408
```

```
# Checking Gradient Boosting accuracy
train_pred = gb.predict(X_train)
test_pred = gb.predict(X_test)

print("Training Accuracy :", accuracy_score(y_train, train_pred))
print("Testing Accuracy :", accuracy_score(y_test, test_pred))
```

```
Training Accuracy : 0.9099756690997567
Testing Accuracy : 0.7684729064039408
```

Activity 1.5: Save the Selected Model

The trained Random Forest model is saved using Joblib as: rdf.pkl

The trained StandardScaler is saved as: scale1.pkl

These files are later loaded into the Flask application to perform real-time loan approval predictions.

```
# Saving the Random Forest model

import pickle

pickle.dump(rf, open('rdf.pkl', 'wb'))
✓ 0.0s

# Saving the Standard Scaler

pickle.dump(sc, open('scale1.pkl', 'wb'))
✓ 0.0s
```

Milestone 2: Flask Backend Development

Description

This milestone focuses on developing the backend of the Smart Lender application using the Flask framework. The backend connects the web interface with the trained Machine Learning model, processes user input, performs loan prediction, and returns the prediction result to the user.

Activity 2.1: Load the Trained Model and Scaler

The Flask application loads the trained Random Forest model (rdf.pkl) and the StandardScaler (scale1.pkl) when the application starts. Loading them once at startup improves performance by avoiding repeated loading during every prediction request.

```
import numpy as np
import pandas as pd
import pickle
from flask import Flask, render_template,
request

model = pickle.load(open("rdf.pkl", "rb"))
scaler = pickle.load(open("scale1.pkl", "rb"))
```

Activity 2.2: Create Flask Application

A Flask application instance is created to manage HTTP requests and responses. Flask acts as the communication layer between the frontend and the machine learning model.

```
app = Flask(__name__)
```

Activity 2.3: Define Application Routes

Three main routes are defined in the application.

Home Route (/)

Displays the Home page of the Smart Lender application.

Prediction Route (/predict)

Displays the loan prediction form where users enter applicant details.

Submit Route (/submit)

Receives user input through a POST request, performs prediction, and displays the result.

```
# Home page

@app.route("/")
def home():
    return render_template("home.html")

# Prediction page

@app.route("/predict")
def predict():
    return render_template("predict.html")

# Prediction logic

@app.route("/submit", methods=["POST"])
def submit():
```

Activity 2.4: Display Prediction Result

The prediction result is checked.

If the prediction value is 1, the application displays "Loan Will be Approved".

Otherwise, it displays "Loan Will Not be Approved".

Finally, the prediction result is sent to the submit.html page and displayed to the user.

```
if prediction[0] == 1:
    result = "Loan Will be Approved"
else:
    result = "Loan Will Not be Approved"

return render_template("submit.html",
    result=result)
```

Milestone 3: Conclusion

The Smart Lender – Loan Approval Prediction Using Machine Learning project successfully demonstrates how Machine Learning can be used to automate the loan approval process. By analyzing applicant details such as income, education, credit history, loan amount, and other relevant features, the system predicts whether a loan application is likely to be approved or rejected.

The project includes a complete end-to-end workflow, starting from data preprocessing and model training in the Jupyter Notebook (main.ipynb) to deploying the trained Random Forest model through a Flask web application. The application provides a simple and user-friendly interface where users can enter applicant information and instantly receive a loan approval prediction.

Among the machine learning algorithms evaluated, Random Forest achieved the best performance and was selected as the final model. The trained model (rdf.pkl) and the fitted StandardScaler (scale1.pkl) were integrated into the Flask backend to ensure accurate and consistent predictions.

Overall, the Smart Lender system helps reduce manual effort, improves the speed of loan evaluation, and supports more consistent decision-making.

Hardware Requirements

Processor: Intel i3 or above **RAM:**

4 GB (8 GB recommended) **Storage:**

10 GB free space

System Type: 64-bit OS

Display: 1366×768 or higher

Skills Required:

NumPy
Pandas
Scikit-learn
Matplotlib (Python Package)
SciPy
Machine Learning
Seaborn
Flask (Web Framework)
Python (Programming Language)

Team Details:

Team ID: xxx

Team Members

Neeli Likhith Madhav
Neeharika Veluvolu
Sarani Phanisree Nowbottu
Raunak Rai
Kalluri Rishendra

Conclusion:

Smart Leader is a Machine Learning based loan eligibility prediction platform built with Python, Flask, and Scikit-learn/XGBoost. It streamlines credit evaluation by analyzing applicant attributes and delivering instant, data-driven eligibility predictions through a user-friendly web interface. The project, which covered data collection, visualization, preprocessing, model training, and Flask deployment, demonstrates how targeted machine learning and a structured development framework can help financial institutions reduce evaluation time, minimize non-performing assets, and improve overall lending confidence.

Looking ahead, Smart Leader offers opportunities for expansion, such as cloud deployment on IBM Cloud, integrating additional applicant risk signals, and adding user authentication to maintain prediction history.